

Assignment 2: Speaker Diarization

EE6130: Advanced Topics in Signal Processing

Nishchala Mukku (EE24S004)

October 28, 2024

1 Question

Tasks for Speaker Diarization:

- Write your own clustering algorithm
- Vary the chunk size from 0.5s to 3s in steps of 0.5s and analyze the results.

Write a two page report on your findings

2 Solution - My Work

Google Colab Notebook link: [Link](#)

2.1 Own clustering algorithm

The clustering algorithm used here is **Agglomerative clustering**. It is used to group data points based on their similarity. It's a bottoms-up approach i.e, merges the neighboring clusters as one upon similarity being detected, using distance metric and linkage criteria. The steps involved in the Agglomerative clustering are as follows,

- if there are N data-points, you will have N initial clusters.
- Compute distance between pairs of clusters.
- Find the closest clusters based on distance metric and linkage criteria and merge them into a single cluster.
- Re-calculate distance between new cluster and all other clusters.
- Repeat until the stopping criteria which is numbers of clusters is reached.

To get started, I defined a class definition which contains following member functions,

- Initialization (`__init__`): The *AgglomerativeClustering* class initializes the number of clusters (`n_clusters`) and the linkage criterion (`linkage`), which can be complete, single, or average.
- Fit method: This is the core of the algorithm. It starts by treating each point as a single cluster and calculates pairwise distances between all points. Then, iteratively, it merges the closest clusters until the desired number of clusters is reached.
- `compute_initial_distances()`: This method computes a distance matrix between all data points using the Euclidean distance.
- `find_closest_clusters()`: It finds the two closest clusters to be merged based on the selected linkage criterion (used average).
- `merge_clusters()`: It merges two clusters by appending the points of one cluster to the other.

```

1 class AgglomerativeClustering:
2     def __init__(self, n_clusters=2):
3         self.n_clusters = n_clusters
4
5     def fit(self, X):
6         self.clusters = [[] for i in range(len(X))]
7         self.distances = self.compute_initial_distances(X)
8         # since it's a bottom up approach, we do the merging until we reach the number of clusters required criteria
9         while(len(self.clusters) > self.n_clusters):
10             c1, c2 = self.find_closest_clusters()
11             self.merge_clusters(c1, c2)
12             return self.clusters
13
14         # euclidean distance between points
15         # X contains all the points and clusters (not initially) as a matrix form
16     def compute_initial_distances(self, X):
17         n = len(X)
18         distances = np.zeros((n, n))
19         for i in range(n):
20             for j in range(i+1, n): # since the matrix will be symmetric, we can initialise the distance of (i, j) entity to (j, i)
21                 distances[i, j] = euclidean(X[i], X[j])
22                 distances[j, i] = distances[i, j]
23         return distances
24
25     # Next, finding the closest clusters based on the distance
26     def find_closest_clusters(self):
27         min_dist = np.inf
28         c1, c2 = None, None
29         # clusters
30         for i in range(len(self.clusters)):
31             for j in range(i+1, len(self.clusters)):
32                 dist = self.compute_distance_between_clusters(self.clusters[i], self.clusters[j])
33                 if dist < min_dist:
34                     min_dist = dist
35                     c1, c2 = i, j
36         return c1, c2
37
38     # distance between clusters can be computed using various methods. Here, we are taking the average distances between all points in the two clusters, i.e., average linkage.
39     def compute_distance_between_clusters(self, c1, c2):
40         avg = np.mean([self.distances[i, j] for i in c1 for j in c2])
41         return avg
42
43     # After finding the closest clusters, we need to merge them as one cluster
44     # c1 and c2 are indices of the clusters in the matrix consisting of clusters
45     def merge_clusters(self, c1, c2):
46         self.clusters[c1].extend(self.clusters[c2])
47         del self.clusters[c2]

```

Figure 1: Class definition for Agglomerative clustering from scratch

This algorithm working verified using a test data of random 100 points. This algorithm can be made better if

2.2 Speaker diarization analysis

Speaker diarization is identifying and segmenting different speakers within an audio recording. This involved dividing the audio into segments and then identifying and assigning the speaker label. There are various steps involved in the process,

```
7 # Print the transcription results
8 print("For standard segment size: ")
9 print(transcription_results)
10 print(f"Transcription results saved at: {save_path}")
```

	Start	End	Speaker
0	0:00:00	0:00:04	SPEAKER 2
1	0:00:04	0:00:27	SPEAKER 1
2	0:00:27	0:00:31	SPEAKER 2
3	0:00:31	0:00:54	SPEAKER 1
4	0:00:54	0:00:57	SPEAKER 2
5	0:00:57	0:01:21	SPEAKER 1
6	0:01:21	0:01:26	SPEAKER 2
7	0:01:26	0:01:56	SPEAKER 1

Transcription results saved at: /content/TEST-1.csv

Figure 2: Original transcription results

```
26 # Function to merge segments to a target duration
27 def merge_segments(segments, target_duration):
28     merged_segments = []
29     current_segment = None
30
31     for segment in segments:
32         start = segment["start"]
33         end = segment["end"]
34
35         if current_segment is None:
36             current_segment = {"start": start, "end": end}
37         elif (current_segment["end"] - current_segment["start"]) + (end - start) <= target_duration:
38             current_segment["end"] = end
39         else:
40             merged_segments.append(current_segment)
41             current_segment = {"start": start, "end": end}
42
43     if current_segment:
44         merged_segments.append(current_segment)
45
46     return merged_segments
```

Figure 3: Merged segments code

- Voice activity Detection (VAD): Detect the voiced part of the signal and filter out non speech segments like silence, noise or background music.
- Speaker segmentation: Audio is segmented based on changes in speaker identity, even if content is continuous.
- Speaker clustering: Segments are grouped or clustered together based on similarities in speaker voice characteristics. Each cluster correspond to a unique speaker.
- Labelling: Labelling each segment with corresponding speaker based on the clusters.

We are require to make the chunk size from 0.5s to 3s in the steps of 0.5s. For this I added an extra function to merged the segments if they are less than the chunk size. This can be seen in the figure 3. The original transcription results are as given in 2.

Choosing an appropriate window for chunks is important cause difference in the performance of the process and can affect the clustering algorithm. Smaller segments can improve word boundary and improve diarization in highly dynamic conversations where speakers change frequently. Larger segments can handle sustained speaker turns better, making them useful for longer, uninterrupted speaking periods but at the cost of missing subtle switches.

Usually speaker diarization are evaluated based on Diarization Error Rate (DER) but since we don't have reference annotations, this metric isn't useful. We have to see how the model behaved for no segments breakages and comparing the silhouette score for the clusters formed in each case.

For the given range of 0.5s to 3s, the following transcription results are obtained,

```
Transcribing with chunk size: 0.5 seconds

Audio duration: 0:02:01
Silhouette Score for 0.5 second segment size: 0.4208
Transcription Results:
      Start      End      Speaker
0  0:00:00  0:00:04  SPEAKER 2
1  0:00:04  0:00:07  SPEAKER 1
2  0:00:07  0:00:13  SPEAKER 1
3  0:00:13  0:00:21  SPEAKER 1
4  0:00:21  0:00:27  SPEAKER 1
5  0:00:27  0:00:31  SPEAKER 2
6  0:00:31  0:00:37  SPEAKER 1
7  0:00:37  0:00:44  SPEAKER 1
8  0:00:44  0:00:51  SPEAKER 1
9  0:00:51  0:00:54  SPEAKER 1
10 0:00:54  0:00:57  SPEAKER 2
11 0:00:57  0:01:03  SPEAKER 1
12 0:01:04  0:01:08  SPEAKER 1
13 0:01:09  0:01:14  SPEAKER 1
14 0:01:14  0:01:21  SPEAKER 1
15 0:01:21  0:01:26  SPEAKER 2
16 0:01:26  0:01:34  SPEAKER 1
17 0:01:34  0:01:38  SPEAKER 1
18 0:01:38  0:01:44  SPEAKER 1
19 0:01:44  0:01:49  SPEAKER 1
20 0:01:49  0:01:52  SPEAKER 1
21 0:01:52  0:01:56  SPEAKER 1
22 0:01:56  0:02:01  SPEAKER 1
Moviepy - Building video /content/videoplayback_label_duration0.5.mp4.
MoviePy - Writing audio in videoplayback_label_duration0.5TEMP_MPY_wvf_snd.mp3
MoviePy - Done.
Moviepy - Writing video /content/videoplayback_label_duration0.5.mp4

Moviepy - Done !
Moviepy - video ready /content/videoplayback_label_duration0.5.mp4

unk size: 1.0 seconds
```

Figure 4: Results for chunk size = 0.5s

```
Moviepy - Done !
Moviepy - video ready /content/videoplayback_label_duration2.5.mp4

Transcribing with chunk size: 3.0 seconds

Audio duration: 0:02:01
Silhouette Score for 3.0 second segment size: 0.4208
Transcription Results:
      Start      End      Speaker
0  0:00:00  0:00:04  SPEAKER 2
1  0:00:04  0:00:07  SPEAKER 1
2  0:00:07  0:00:13  SPEAKER 1
3  0:00:13  0:00:21  SPEAKER 1
4  0:00:21  0:00:27  SPEAKER 1
5  0:00:27  0:00:31  SPEAKER 2
6  0:00:31  0:00:37  SPEAKER 1
7  0:00:37  0:00:44  SPEAKER 1
8  0:00:44  0:00:51  SPEAKER 1
9  0:00:51  0:00:54  SPEAKER 1
10 0:00:54  0:00:57  SPEAKER 2
11 0:00:57  0:01:03  SPEAKER 1
12 0:01:04  0:01:08  SPEAKER 1
13 0:01:09  0:01:14  SPEAKER 1
14 0:01:14  0:01:21  SPEAKER 1
15 0:01:21  0:01:26  SPEAKER 2
16 0:01:26  0:01:34  SPEAKER 1
17 0:01:34  0:01:38  SPEAKER 1
18 0:01:38  0:01:44  SPEAKER 1
19 0:01:44  0:01:49  SPEAKER 1
20 0:01:49  0:01:52  SPEAKER 1
21 0:01:52  0:01:56  SPEAKER 1
22 0:01:56  0:02:01  SPEAKER 1
Moviepy - Building video /content/videoplayback_label_duration3.0.mp4.
MoviePy - Writing audio in videoplayback_label_duration3.0TEMP_MPY_wvf_snd.mp3
MoviePy - Done.
Moviepy - Writing video /content/videoplayback_label_duration3.0.mp4

Moviepy - Done !
Moviepy - video ready /content/videoplayback_label_duration3.0.mp4
```

Figure 5: Results for chunk size = 3s

Similar results have been produced for the remaining sets of iterations til 3s. The following is my conclusion,

- The silhouette score of 0.4208 remains consistent across all chunk sizes, indicating that clustering quality is relatively stable regardless of segment size, till 3s.
- Across all the chunks, the diarization results are consistent and identify the speakers correctly. This consistency indicates that the model is fairly accurate in identifying speaker boundaries and that chunk size has minimal effect on the model's ability to detect speaker changes in this particular audio file.
- We can observe that the clustering is where the segments are merged into

```
✓ 13m Silhouette Score for 9.5 second segment size: 0.4394
Transcription Results:
      Start      End      Speaker
0  0:00:00  0:00:07  SPEAKER 1
1  0:00:07  0:00:13  SPEAKER 1
2  0:00:13  0:00:21  SPEAKER 1
3  0:00:21  0:00:27  SPEAKER 1
4  0:00:27  0:00:31  SPEAKER 2
5  0:00:31  0:00:37  SPEAKER 1
6  0:00:37  0:00:44  SPEAKER 1
7  0:00:44  0:00:51  SPEAKER 1
8  0:00:51  0:00:57  SPEAKER 1
9  0:00:57  0:01:03  SPEAKER 1
10 0:01:04  0:01:14  SPEAKER 1
11 0:01:14  0:01:21  SPEAKER 1
12 0:01:21  0:01:26  SPEAKER 2
13 0:01:26  0:01:34  SPEAKER 1
14 0:01:34  0:01:38  SPEAKER 1
15 0:01:38  0:01:44  SPEAKER 1
16 0:01:44  0:01:52  SPEAKER 1
17 0:01:52  0:02:01  SPEAKER 1
Moviepy - Building video /content/videoplayback_label_duration9.5.mp4.
MoviePy - Writing audio in videoplayback_label_duration9.5TEMP_MPY_wvf_snd.mp3
MoviePy - Done.
Moviepy - Writing video /content/videoplayback_label_duration9.5.mp4

Moviepy - Done !
Moviepy - video ready /content/videoplayback_label_duration9.5.mp4
```

Figure 6: Results for chunk size = 9.5

longer segments when the same speaker continues speaking across adjacent chunks. This behavior is observed to be independent of the chunk size.

- Upon further experimenting with the chunk size till 20s, we can observe that upto 8.5s, the silhouette scores are consistent and beyond 14s, the scores drop indicating potential errors in clustering. Scores between 6 and 8.5 seconds are generally more consistent (approx. 0.41–0.42).
- Up to an 8.5-second chunk size, the speaker labels seem accurate and consistent with fewer changes. With chunk sizes of 14+ seconds, the clustering becomes less stable, with segments sometimes attributed to the wrong speaker, shown by more Speaker 2 labels dominating longer segments.
- The speaker diarization accuracy also reduces when the chunk size increases. The speakers aren't identified correctly.

The results are available in the Google Colab notebook provided. The video files having the speaker labels written upon them are also attached in the .zip file submitted along with this report.

```

Moviepy - video ready /content/videoplayback_label_duration15.5.mp4
13m 13m
Transcribing with chunk size: 17.5 seconds

Audio duration: 0:02:01
Silhouette Score for 17.5 second segment size: 0.1228
Transcription Results:
      Start      End      Speaker
0 0:00:00 0:00:13 SPEAKER 1
1 0:00:13 0:00:27 SPEAKER 1
2 0:00:27 0:00:44 SPEAKER 1
3 0:00:44 0:00:57 SPEAKER 1
4 0:00:57 0:01:14 SPEAKER 2
5 0:01:14 0:01:26 SPEAKER 1
6 0:01:26 0:01:38 SPEAKER 2
7 0:01:38 0:01:52 SPEAKER 2
8 0:01:52 0:02:01 SPEAKER 2
Moviepy - Building video /content/videoplayback_label_duration17.5.mp4.
MoviePy - Writing audio in videoplayback_label_duration17.5TEMP_MPY_wvf_snd.mp3
MoviePy - Done.
Moviepy - Writing video /content/videoplayback_label_duration17.5.mp4

Moviepy - Done !
Moviepy - video ready /content/videoplayback_label_duration17.5.mp4

```

Figure 7: Results for chunk size = 17.5

```

Moviepy - video ready /content/videoplayback_label_duration17.5.mp4
13m 13m
Transcribing with chunk size: 19.5 seconds

Audio duration: 0:02:01
Silhouette Score for 19.5 second segment size: 0.2907
Transcription Results:
      Start      End      Speaker
0 0:00:00 0:00:13 SPEAKER 1
1 0:00:13 0:00:31 SPEAKER 1
2 0:00:31 0:00:44 SPEAKER 1
3 0:00:44 0:01:03 SPEAKER 1
4 0:01:04 0:01:21 SPEAKER 1
5 0:01:21 0:01:38 SPEAKER 1
6 0:01:38 0:01:56 SPEAKER 1
7 0:01:56 0:02:01 SPEAKER 2
Moviepy - Building video /content/videoplayback_label_duration19.5.mp4.
MoviePy - Writing audio in videoplayback_label_duration19.5TEMP_MPY_wvf_snd.mp3
MoviePy - Done.
Moviepy - Writing video /content/videoplayback_label_duration19.5.mp4

Moviepy - Done !
Moviepy - video ready /content/videoplayback_label_duration19.5.mp4

```

Figure 8: Results for chunk size = 19.5

3 Conclusion

- Agglomerative clustering with distance metric as Euclidean distance and average linkage has been implemented.
- Chunking of segments is recommended only if the average speaker changes happens at constant value. For frequent changes, it is recommended to follow the procedure without any chunking to avoid mix-up of clusters and incorrect prediction.
- Improper chunk size can effect final results.
- Smaller segments/chunks get merged during clustering process when the same speaker continues to speak across adjacent chunks,independent of chunk size.